

Universidad Tecnológica de la Mixteca

Analizador Léxico en Flex

para el lenguaje C++

CompiladorEq3 — Asignación 2

Curso: Compiladores
Profesor: David Martínez Torres
Programa: Ingeniería en Computación, quinto semestre

Equipo 3

Arenas Rojas Ivri Nitzani
Bautista Jiménez Israel
Hernández Torija Itzel Amairani
Juárez Jiménez Yarelli Sinai
López López Xochitl Nerina
Quiroz Martínez Maybelin

Oaxaca, México

Introducción

En la primera entrega el equipo construyó un analizador léxico para C++ en Python: el Autómata Finito Determinista (AFD) se modeló en JFLAP y se tradujo, estado por estado, a las clases `AFDAlexico` y `AnalizadorLexico`. Este informe documenta la segunda entrega (Asignación 2), en la que el mismo analizador léxico se implementa con **Flex**, la herramienta de generación automática de analizadores léxicos que se estudia en el curso.

Con Flex ya no se programa el autómata a mano: se escriben las expresiones regulares de cada token en un archivo `lexer.l`, y Flex construye el autómata automáticamente y genera un programa en C (`lex.yy.c`) que lo recorre. Ese programa se compila con `gcc` y se ejecuta desde la terminal sobre un programa fuente en C++, o desde la interfaz gráfica de la primera entrega, ampliada con un panel para este propósito. Produce tres salidas: la tira de tokens, la tabla de símbolos y la tabla de errores léxicos, con el mismo formato y los mismos mensajes que la versión en Python.

El trabajo se organizó en ocho partes: (1) traducción de las expresiones regulares de la primera entrega a la notación de Flex, (2) relación entre esas expresiones, el AFD de JFLAP y el autómata que Flex construye internamente, (3) descripción del algoritmo de reconocimiento, (4) escritura de `lexer.l` (reglas, tira de tokens, tabla de símbolos y manejo de errores), (5) compilación y ejecución en la terminal, (6) integración con la interfaz gráfica, (7) pruebas sobre los archivos de la primera entrega, y (8) distribución de responsabilidades entre los seis integrantes del equipo.

1 Análisis

1.1 ¿Qué es Flex?

Flex (*Fast Lexical Analyzer Generator*) es la versión libre de la herramienta `lex` de Unix. Recibe un archivo de especificación con expresiones regulares y la acción en C que se ejecuta cuando cada una se reconoce, y genera el código fuente de un analizador léxico completo. Internamente sigue los mismos pasos que se estudian en el curso para construir un AFD a partir de expresiones regulares:

1. Convierte cada expresión regular en un Autómata Finito No determinista (AFN), mediante la construcción de Thompson.
2. Une todos los AFN en uno solo, con un estado inicial común.
3. Convierte ese AFN en un AFD mediante la construcción de subconjuntos.
4. Minimiza y comprime las tablas del AFD y las escribe en `lex.yy.c`, junto con la función `yylex()`, que recorre la entrada carácter por carácter aplicando ese autómata.

1.2 Herramientas utilizadas

1.3 Estructura de un archivo de Flex

Un archivo `.l` tiene tres secciones separadas por `%%`:

```
%{
   /* codigo C que se copia tal cual: includes, variables, prototipos */
%}
```

Tabla 1: Herramientas utilizadas en la Asignación 2.

Herramienta	Uso
Flex 2.6.4	Genera `lex.yy.c` a partir de `lexer.l` (Linux y macOS).
gcc 13.3	Compila `lex.yy.c` y produce el ejecutable `analizador_lexico`.
win_flex (WinFlexBison 2.5.25, flex 2.6.4)	Flex para Windows; se invoca con la opción `--wincompat`.
TinyCC 0.9.27	Compilador de C portátil para Windows de 64 bits; va integrado en el `.exe` de la interfaz.
Python 3 + Tkinter	Interfaz gráfica que compila y ejecuta el analizador de Flex (módulo `compilador_flex.py`).
JFLAP	Modelado del AFD de la primera entrega, usado aquí como referencia y punto de partida.

```
   /* definiciones: nombres para expresiones regulares y opciones */
%%
   /* reglas:  expresion_regular   { accion en C } */
%%
   /* codigo de usuario: funciones auxiliares y main() */
```

1.4 Expresiones regulares en notación de Flex

Las expresiones regulares de la primera entrega (`expresiones_regulares.pdf`) se tradujeron a la sintaxis de Flex. La Tabla 2 muestra cada definición tal como aparece en `lexer.l`.

Tabla 2: Expresiones regulares de la especificación y su versión en Flex.

Nombre	Especificación	Flex (`lexer.l`)
letra	`[A-Za-z]`	`letra [A-Za-z]`
dígito	`[0-9]`	`digito [0-9]`
identificador	letra(letra | dígito | _)*	`{letra}({letra}|{digito}|_)*`
entero (`nint`)	dígito$^+$	`{digito}+`
real (`nfloat`)	dígito$^+$. dígito$^+$	`{digito}+\.{digito}+`
escape	\(" | \ | n | t | r | ' | 0)	`\\(\"|\\\|n|t|r|'|0)`
cuerpo de cadena (`caracterCadena`)	letra | dígito | caracterEspecial	`[^"\\\n]`
cadena (`literalCadena`)	`"(...)*"`	`\"({caracterCadena}|{escape})*\"`
carácter (`literalCar`)	'(letra | dígito | caracterEspecial)'	`'[^'\n]'`
comentario de línea	`//` seguido del resto de la línea	`"//".*`
comentario de bloque	`/* ... */`	`"/*"([^*]|\*+[^/*])*\*+"/"`

Diferencias con la especificación

Al traducir la especificación a Flex, el equipo tomó cuatro decisiones de diseño, revisadas y aceptadas por el profesor:

- **Cuerpo de cadena.** La especificación enumera los caracteres permitidos uno por uno; en Flex se usa la clase `[^"\\\n]` (cualquier carácter excepto comilla doble, diagonal invertida y salto de línea), que es equivalente pero además acepta cadenas realistas como URLs o correos sin tener que listar cada símbolo. La diagonal invertida solo entra como parte de un `escape`.
- **Carácter.** Se acepta cualquier símbolo excepto la comilla simple y el salto de línea (`'[^'\n]'`), igual que la categoría `CARACTER_SIMPLE` del AFD de la primera entrega.
- **Palabras reservadas.** A *salidaTexto* se agregaron `std`, `printf` y `scanf`, con lo que el conjunto de 42 palabras reservadas de la primera entrega crece a 45 en `lexer.l`.
- **Doble apuntador.** `**` se reconoce como un único token compuesto –el mismo estado q_{20} del AFD que ya distinguía `*`, `*=` y `**`– en lugar de dos tokens de multiplicación consecutivos, para que declaraciones como `int **ptr;` se tokenicen de forma inequívoca.

Las palabras reservadas, los operadores y los delimitadores se escriben directamente como cadenas literales entre comillas en la sección de reglas (por ejemplo `"int"|"float"`), porque no varían: no necesitan una expresión regular, solo una alternativa exacta.

2 Autómatas

2.1 AFD de referencia (JFLAP)

La Figura 1 es el AFD de la primera entrega, modelado en JFLAP y ya validado en el informe anterior. En Flex no se dibuja ni se programa directamente: funciona como especificación de qué tokens existen y qué prefijos son válidos –por ejemplo, que `<` debe seguir siendo un token aunque pueda continuar hacia `<=` o `<<`–. La Tabla 3 relaciona cada familia de estados de aceptación del AFD con la regla de `lexer.l` que la reemplaza.

Tabla 3: Estados de aceptación del AFD y regla equivalente en Flex.

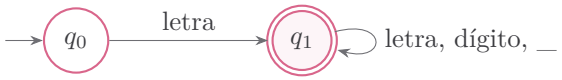
Familia	Estados del AFD	Regla en `lexer.l`
Identificador	q_{35}	`{identificador}`
Números	q_{32} (`nint`), q_{34} (`nfloat`)	`{nint}`, `{nfloat}`
Cadena	$q_9 \to q_{11}$	`{literalCadena}`
Carácter	$q_{14} \to q_{15} \to q_{16}$	`{literalCar}`
Comentarios	q_{25}–q_{31}	`"//".*` y la regla de bloque
Delimitadores	q_1–q_6, q_7, q_{10}, q_{13}	`"{"`, `"}"`, `"("`, …
Suma	q_{54}, q_{55}, q_{56}	`"+"`, `"+="`, `"++"`
Resta y flecha	q_{50}–q_{53}	`"-"`, `"-="`, `"--"`, `"->"`

Familia	Estados del AFD	Regla en `lexer.l`
Producto	q_{18}, q_{19}, q_{20}	`"*"`, `"*="`, `"**"`
División	q_{25}, q_{60}	`"/"`, `"/="`
Relacionales	q_{38}, q_{39}, q_{21}, q_{22}, q_{44}–q_{49}	`"="`, `"=="`, `"!"`, `"!="`, `"<"`, `"<="`, `"<<"`, …
Lógicos y módulo	q_{40}–q_{43}, q_{23}, q_{24}	`"&"`, `"&&"`, `"|"`, `"||"`, `"%"`, `"%="`
Resolución	q_{8}	`"::"`
Otros	q_{37}, q_{57}, q_{58}, q_{59}	`"?"`, `"#"`, `"~"`, `"^"`

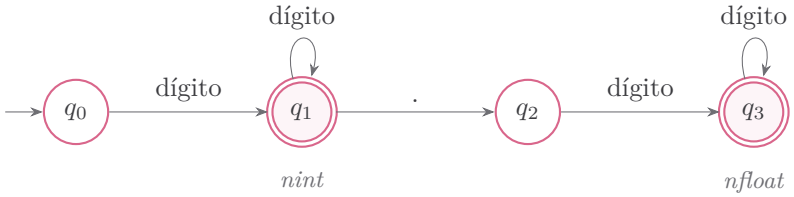
2.2 Autómatas de cada expresión regular

Los siguientes diagramas muestran el autómata que corresponde a cada expresión regular por separado, dibujados a mano para ilustrar la construcción de Thompson. Flex los une en uno solo con la construcción de subconjuntos (sección 2.3).

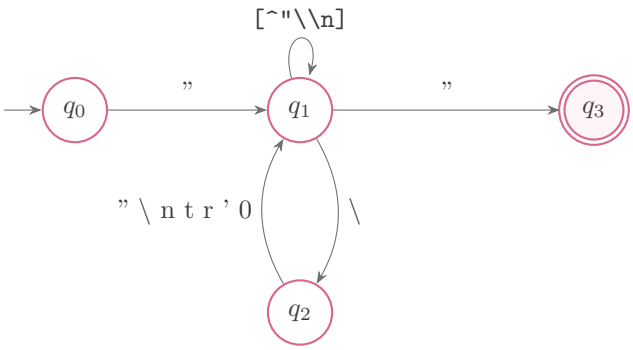
Identificador. `{letra}({letra}|{digito}|_)*`



Números enteros y reales. `{digito}+` *y* `{digito}+\.{digito}+`



Cadena. `\"({caracterCadena}|{escape})*\"`



Carácter. `'[^'\n]'`

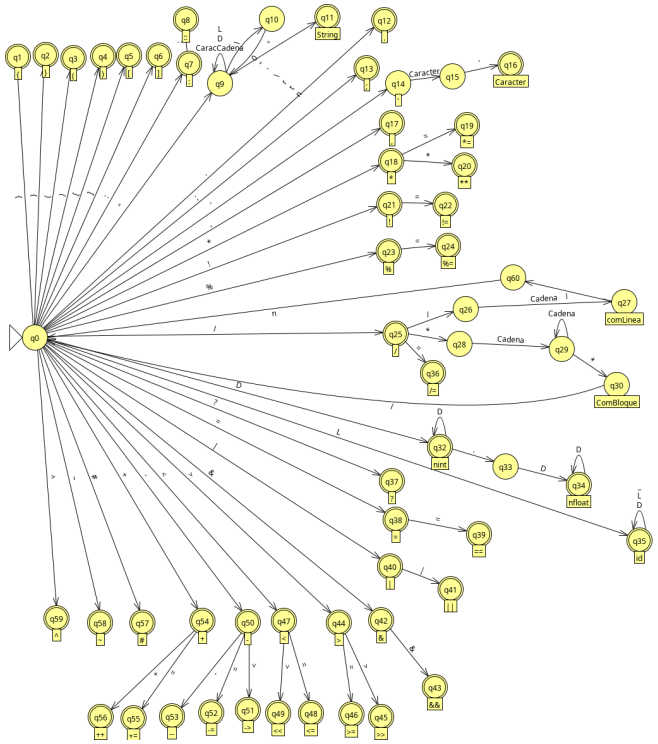
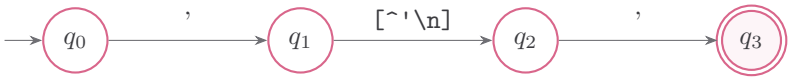
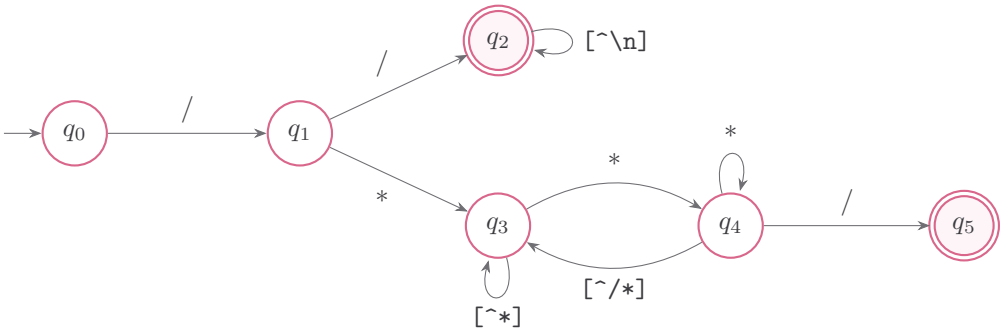


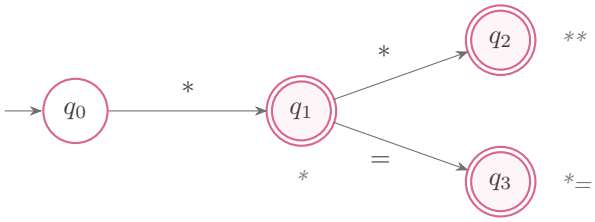
Figura 1: AFD del analizador léxico modelado en JFLAP (primera entrega), usado como referencia para `lexer.l`.



Comentarios. `"//".*` y `"/*"([^*]|\*+[^/*])*\*+"/"`



Operadores compuestos (ejemplo: producto). `"*"`, `"*="`, `"**"`



Como en el AFD de JFLAP, cada prefijo de un operador compuesto es también un estado de aceptación: `*` es un token válido aunque pueda continuar a `*=` o `**`. Los demás operadores (`+`, `-`, `<`, `>`, `=`, `!`, `&`, `|`, `%`, `:`) siguen el mismo patrón, y es justamente lo que permite que la coincidencia más larga funcione sin ambigüedad.

2.3 El AFD que genera Flex

Al ejecutar `flex -v lexer.l`, Flex reporta las estadísticas del autómata que construyó internamente (Tabla 4).

Tabla 4: Estadísticas reportadas por `flex -v lexer.l`.

Concepto	Valor
Estados del AFN (construcción de Thompson)	595
Estados del AFD (construcción de subconjuntos)	201
Reglas	28
Condiciones de arranque (`INITIAL`, `RESTO_LINEA`, `RESTO_ARCHIVO`)	3
Clases de equivalencia de caracteres	58

El AFD que genera Flex (201 estados) es considerablemente más grande que el de JFLAP (60 estados) porque reconoce cada palabra reservada con su propio camino de estados completo, en lugar de reconocer un identificador genérico y consultarlo después en una tabla hash, como hace la versión en Python. Las *clases de equivalencia* agrupan los caracteres que se comportan igual en todas las transiciones del autómata (por ejemplo, todas las letras que no pueden iniciar ninguna palabra reservada); es la misma idea que las categorías de símbolos (`LETRA`, `DIGITO`, …) de la primera entrega, solo que Flex la calcula automáticamente en vez de programarla a mano.

2.4 Reglas para resolver ambigüedades

Cuando varias expresiones regulares coinciden con la entrada en un mismo punto, Flex aplica, en este orden, las dos reglas clásicas de un analizador léxico:

1. **Coincidencia más larga** (*maximal munch*): gana la regla que reconoce el lexema más largo. Así, `<<` es un solo token y no dos `<` consecutivos, y `3.14` es un `nfloat` y no tres tokens (`3`, `.`, `14`).
2. **Primera regla**: si dos reglas reconocen un lexema de exactamente la misma longitud, gana la que aparece primero en el archivo. Por eso las palabras reservadas se escriben *antes* que

`{identificador}` en `lexer.l`: `int` coincide con ambas reglas por igual, y debe ganar la de palabra reservada.

Es exactamente el mismo algoritmo de coincidencia máxima con retroceso que el equipo implementó a mano en `AnalizadorLexico.siguiente_token()` durante la primera entrega; en Flex lo realiza automáticamente la función generada `yylex()`.

3 Descripción del algoritmo

El analizador se ejecuta como un único AFD recorrido por el motor interno de Flex (`yylex()`), con cuatro fases claramente delimitadas en `lexer.l`.

1. Inicialización (`main`)

Antes de analizar, `main()` abre el archivo fuente en C++ recibido como argumento, crea los tres archivos de salida (`tiraTokens.txt`, `tablaSimbolos.txt` y `errores.txt`) e inicializa los contadores de posición (`numLinea = 1`, `numColumna = 1`) y el puntero de la tabla de símbolos (`tSimb = NULL`).

2. Procesamiento de reglas léxicas (macro `YY_USER_ACTION`)

Antes de ejecutar la acción de cualquier regla que haya coincidido, Flex invoca automáticamente la macro `YY_USER_ACTION`. El equipo la usa para llevar la columna exacta de cada lexema:

```
#define YY_USER_ACTION { colInicio = numColumna; numColumna += yyleng; }
```

`colInicio` guarda la columna donde *empieza* el lexema que se acaba de reconocer, y `numColumna` avanza `yyleng` posiciones (el largo del lexema) para que el siguiente token comience en la columna correcta. El salto de línea se maneja aparte, en su propia regla (`numLinea++; numColumna = 1;`), porque `yyleng` no distingue saltos de línea de otros caracteres.

3. Clasificación y precedencia

- **Operadores compuestos antes que simples.** Las reglas de operadores de dos o más caracteres (`++`, `+=`, `::`, `->`, …) aparecen en `lexer.l` *antes* que las de un solo carácter (`+`, `=`, `:`, `-`), aunque en Flex esto es una convención de claridad y no estrictamente necesario: la coincidencia más larga ya garantiza que gane el operador compuesto sin importar el orden.
- **Doble apuntador (`**`).** Para que declaraciones como `int **ptr;` se lean sin ambigüedad, `**` se reconoce como un único token compuesto (no como dos tokens de multiplicación seguidos), en el mismo grupo de reglas que `*` y `*=`.

4. Gestión de identificadores y tabla de símbolos

Cuando la regla `{identificador}` reconoce un lexema, se llama a `agregarSimbolo(yytext)`, que consulta `buscar()` sobre la lista enlazada de la tabla de símbolos. Si el identificador no existe todavía, se reserva memoria dinámica con `malloc` y el nuevo nodo se inserta al *final* de la lista con `insertar()`, para que la tabla de símbolos quede en el mismo orden en que las variables aparecen por primera vez en el programa fuente (sección 4.4).

4 Implementación en Flex (`lexer.l`)

4.1 Sección de definiciones

```
%option noyywrap

%{
typedef struct elemento {        /* nodo de la tabla de simbolos */
    char nombre[256];
    struct elemento *sig;
} ElementoTSimb;

int numLinea = 1, numColumna = 1, colInicio = 1, numErrores = 0;

/* se ejecuta antes de cada accion: guarda la columna del lexema */
#define YY_USER_ACTION { colInicio = numColumna; numColumna += yyleng; }
%}

%x RESTO_LINEA RESTO_ARCHIVO     /* estados para recuperarse de errores */
```

- `%option noyywrap` indica que solo se analiza un archivo de una sola vez, así que no hace falta definir la función `yywrap()` que Flex pediría de otro modo al llegar al final de la entrada.
- `YY_USER_ACTION` es la macro descrita en la sección anterior.
- `%x` declara *condiciones de arranque exclusivas*, que se usan para descartar texto después de un error (sección 4.5).

4.2 Sección de reglas

Las reglas de `lexer.l` siguen el orden de la Tabla 5. Este fragmento muestra una regla de cada grupo:

```
[\t\r\v\f ]+      { /* ignorar espacios */ }
\n                { numLinea++; numColumna = 1; }
"//".*            { /* ignorar comentario de linea */ }

"int"|"float"|"double"|"char"|"bool"|"void"|"string"   { escribirToken(yytext); }
"::"|"**"|"++"|"--"|"+="|"-="|"*="|"/="|"%="|"<="|">="|"=="|"!="|"&&"|"||"|"<<"|">>"
    |"->"
                  { escribirToken(yytext); }

{nfloat}          { escribirTokenValor("nfloat", ""); }
{nint}            { escribirTokenValor("nint", ""); }
{literalCadena}   { escribirTokenValor("literal", ""); }
{identificador}   { escribirTokenValor("id", "\""); agregarSimbolo(yytext); }

\"                { escribirError("\"", "Cadena sin cerrar"); BEGIN(RESTO_LINEA); }
.                 { escribirError(yytext, "Caracter no reconocido"); }
```

Tabla 5: Orden de las reglas en `lexer.l`.

#	Grupo de reglas
1	Espacios y saltos de línea (se descartan; el salto incrementa la línea).
2	Comentarios de línea y de bloque, completos (se descartan).
3	Palabras reservadas (antes que el identificador, por la regla de primera coincidencia).
4	Operadores compuestos y simples.
5	Delimitadores.
6	Números, literales e identificadores.
7	Errores: cadena, carácter o comentario de bloque sin cerrar, y carácter no reconocido.

4.3 Tira de tokens

Cada token se imprime con su número de línea, su lexema y su token, tanto en la consola como en `tiraTokens.txt`. Las palabras reservadas, los operadores y los delimitadores usan el propio lexema como token; los identificadores, números y literales se escriben como `tipo.valor` (los identificadores, además, entre comillas):

```
/* Renglon de la tira de tokens: No linea | lexema | token */
void escribirToken(const char *token) {
    printf("%-9d%-26s %s\n", numLinea, yytext, token);
    if (tiraTokens != NULL)
        fprintf(tiraTokens, "%-9d%-26s %s\n", numLinea, yytext, token);
}

/* Token con valor: tipo.valor (nint.1) o, con comillas, tipo."valor" (id."a") */
void escribirTokenValor(const char *tipo, const char *comillas) {
    printf("%-9d%-26s %s.%s%s%s\n", numLinea, yytext, tipo, comillas, yytext,
        comillas);
    if (tiraTokens != NULL)
        fprintf(tiraTokens, "%-9d%-26s %s.%s%s%s\n",
                numLinea, yytext, tipo, comillas, yytext, comillas);
}
```

4.4 Tabla de símbolos

La tabla de símbolos es una lista ligada dinámica de identificadores (`ElementoTSimb`). Cada vez que se reconoce un identificador, `buscar()` la recorre secuencialmente; si el nombre no existe todavía, `insertar()` lo agrega al *final* de la lista, para que la tabla quede en el mismo orden en que las variables aparecen por primera vez en el programa:

```
void insertar(ElementoTSimb **tSimb, ElementoTSimb *nuevo) {
    nuevo->sig = NULL;
    while (*tSimb != NULL)
```

Tabla 6: Formato de la columna *token* según el tipo de lexema.

Tipo	Lexema	Token
Palabra reservada	`int`	`int`
Operador / delimitador	`=`	`=`
Identificador	`a`	`id."a"`
Entero	`1`	`nint.1`
Real	`3.14`	`nfloat.3.14`
Cadena	`"hola"`	`literal."hola"`
Carácter	`'x'`	`literal.'x'`

```
        tSimb = &(*tSimb)->sig;      /* avanzar hasta el ultimo nodo */
    *tSimb = nuevo;
}

ElementoTSimb* buscar(ElementoTSimb *tSimb, const char nombre[]) {
    while (tSimb != NULL && strcmp(nombre, tSimb->nombre) != 0)
        tSimb = tSimb->sig;
    return tSimb;                    /* NULL si no se encontro */
}

void agregarSimbolo(const char nombre[]) {
    temp = buscar(tSimb, nombre);
    if (temp == NULL) {
        nuevo = (ElementoTSimb*) malloc(sizeof(ElementoTSimb));
        strncpy(nuevo->nombre, nombre, sizeof(nuevo->nombre) - 1);
        nuevo->nombre[sizeof(nuevo->nombre) - 1] = '\0';
        insertar(&tSimb, nuevo);
    }
}
```

Al terminar el análisis, `imprimirTablaSimbolos()` recorre la lista una sola vez y numera cada identificador único, tanto en consola como en `tablaSimbolos.txt`.

4.5 Manejo de errores

El analizador léxico no se detiene ante un error: lo registra y sigue escaneando el resto del programa, exactamente como hacía la versión en Python.

Detección

Toda secuencia de caracteres que no coincide con ninguna expresión regular anterior cae en la regla de recuperación `.` (un punto, que en Flex significa *cualquier carácter individual*), es decir, un símbolo que no pertenece al alfabeto reconocido por ninguna otra regla del lenguaje.

Aislamiento y reporte

En lugar de abortar el análisis, se llama a `escribirError(caracter, mensaje)`, que reporta el fallo por dos canales a la vez: la salida estándar de errores (`stderr`) y el archivo `errores.txt`, con la línea, la columna exacta de inicio, el carácter causante y un mensaje descriptivo del tipo de error:

```
/* Renglon de la tabla de errores: # | Linea | Columna | Caracter | Descripcion */
void escribirError(const char *caracter, const char *mensaje) {
    numErrores++;
    fprintf(stderr, "%-5d%-8d%-10d%-11s%s\n", numErrores, numLinea, colInicio,
        caracter, mensaje);
    if (errores != NULL)
        fprintf(errores, "%-5d%-8d%-10d%-11s%s\n", numErrores, numLinea, colInicio,
            caracter, mensaje);
}
```

Continuidad

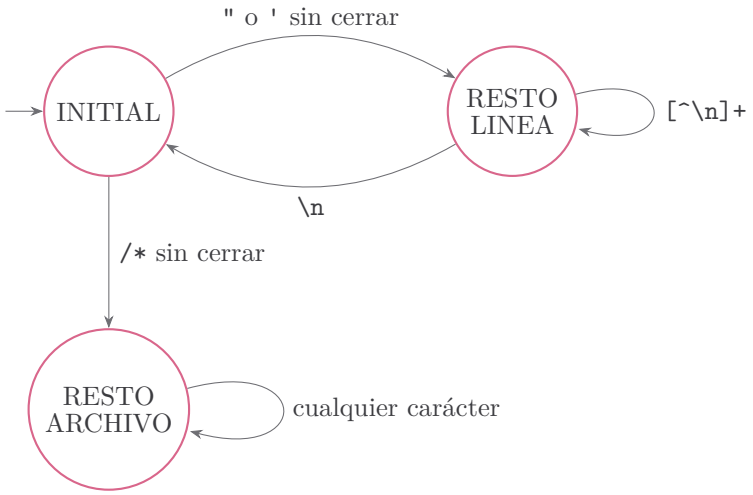
Tras aislar el carácter o la construcción no válida, el puntero de lectura sigue avanzando de forma automática (es el propio motor de `yylex()` el que consume el lexema de cada regla), sin corromper los contadores de línea y columna. Así, un solo análisis puede reportar varios errores léxicos sin detenerse en el primero.

Las reglas de cadena, carácter y comentario de bloque *completos* son más largas que las de error, así que –por coincidencia más larga– estas últimas solo se aplican cuando el literal o el comentario no llegan a cerrarse (Tabla 7).

Tabla 7: Errores léxicos y su recuperación en `lexer.l`.

Mensaje	Regla	Recuperación
Caracter no reconocido	`.`	Se descarta ese único carácter y el análisis continúa con el siguiente.
Cadena sin cerrar	`\"`	`BEGIN(RESTO_LINEA)`: se descarta el resto de la línea.
Caracter sin cerrar	`'`	`BEGIN(RESTO_LINEA)`: se descarta el resto de la línea.
Comentario de bloque sin cerrar	`"/*"`	`BEGIN(RESTO_ARCHIVO)`: se descarta el resto del archivo.

Lo que se descarta durante la recuperación no aparece en la tira de tokens ni en la tabla de símbolos: por ejemplo, las palabras que quedan dentro de una cadena sin cerrar no se registran como identificadores, porque la condición de arranque cambia antes de que Flex vuelva a intentar reconocer tokens normales. El cambio entre condiciones de arranque se comporta como el siguiente autómata:



A diferencia de la primera entrega en Python –donde un comentario de bloque sin cerrar no llegaba a reportarse como error–, en Flex esta situación sí se detecta y se registra explícitamente, gracias a que la condición `RESTO_ARCHIVO` intercepta cualquier `/*` que nunca encuentra su `*/`.

4.6 Función `main()`

`main()` abre el archivo fuente recibido como argumento (o `prueba.cpp` si no se indica ninguno), crea los tres archivos de salida, llama a `yylex()` –que analiza todo el archivo aplicando las reglas anteriores– e imprime la tabla de símbolos al final:

```
int main(int argc, char *argv[]) {
    yyin = fopen(argc > 1 ? argv[1] : "prueba.cpp", "r");
    tiraTokens = fopen("tiraTokens.txt", "w");
    errores = fopen("errores.txt", "w");
    tablaSimbolosArchivo = fopen("tablaSimbolos.txt", "w");

    yylex();
    imprimirTablaSimbolos(tSimb);

    fclose(yyin); fclose(tiraTokens);
    fclose(errores); fclose(tablaSimbolosArchivo);
    return 0;
}
```

5 Compilación y ejecución en la terminal

5.1 Linux y macOS

1. Instalar las herramientas: en Ubuntu/Debian, `sudo apt install flex gcc`; en macOS, `xcode-select --install`.
2. Generar el código C con Flex:

```
$ flex lexer.l            # genera lex.yy.c
```

3. Compilar con gcc:

```
$ gcc -o analizador_lexico lex.yy.c
```

4. Ejecutar sobre un programa fuente:

```
$ ./analizador_lexico ejemplo.cpp
```

5.2 Windows

En Windows se usan `win_flex` y TinyCC en lugar de `flex` y `gcc`. La opción `--wincompat` hace que `lex.yy.c` incluya `<io.h>` en lugar de `<unistd.h>`, que no existe en Windows:

```
> win_flex --wincompat lexer.l
> tcc -o analizador_lexico.exe lex.yy.c
> analizador_lexico.exe ejemplo.cpp
```

El `AnalizadorLexico.exe` de la interfaz trae ambas herramientas integradas en una carpeta `herramientas/`, así que en Windows no es necesario instalar nada para compilar y ejecutar `lexer.l`.

5.3 Ejemplo de ejecución

Para el programa `ejemplo.cpp`:

```
int main(){
int a, b = 1;

a = b + 2;
}
```

la ejecución en la terminal produce:

```
No linea lexema                    token
1        int                       int
1        main                      main
1        (                         (
1        )                         )
1        {                         {
2        int                       int
2        a                         id."a"
2        ,                         ,
2        b                         id."b"
2        =                         =
2        1                         nint.1
2        ;                         ;
4        a                         id."a"
4        =                         =
4        b                         id."b"
4        +                         +
4        2                         nint.2
4        ;                         ;
5        }                         }

Tabla de simbolos
#    Identificador
```

```
1    a
2    b
```

Además se crean, en la carpeta donde se ejecutó el programa, los archivos `tiraTokens.txt`, `tablaSimbolos.txt` y `errores.txt` con el mismo contenido.

5.4 Ejemplo con errores

Para `ErroresLexicos.cpp` –el mismo archivo con cuatro errores a propósito de la primera entrega–, el archivo `errores.txt` contiene:

```
#    Línea   Columna   Carácter   Descripción
1    7       15        @          Caracter no reconocido
2    8       13        $          Caracter no reconocido
3    9       16        "          Cadena sin cerrar
4    10      14        '          Caracter sin cerrar
```

A pesar de los cuatro errores, el analizador reconoce los 41 tokens válidos del archivo, incluidos los de las líneas posteriores al último error. Es exactamente el mismo resultado que reportó la versión en Python en la primera entrega.

6 Integración con la interfaz gráfica

Al iniciar, la interfaz muestra el logo de la Universidad como pantalla principal. En el menú *Analizadores*, la opción *Analizador léxico* despliega un submenú con dos entradas (Figura 2): *Analizador léxico_Flex*, que muestra la pantalla del analizador en Flex, y *Analizador léxico_AFD*, que muestra la del analizador de la primera entrega. Así, cada analizador tiene su propia pantalla, con su propio editor de código fuente.

La interfaz de la primera entrega se amplió con el panel *Analizador en Flex* (Figura 3), con tres botones:

- **Abrir archivo flex**: abre `lexer.l` en una ventana de texto donde se puede editar y guardar, sin salir de la interfaz.
- **Compilar**: ejecuta `flex` y el compilador de C sobre `lexer.l`, y muestra la salida de ambos comandos.
- **Ejecutar analizador léxico (Flex)**: corre el ejecutable compilado sobre el programa cargado en el editor de la interfaz, y abre tres ventanas con la tira de tokens, la tabla de símbolos y la tabla de errores (figuras 4–6).

El módulo `Modulos/compilador_flex.py` hace de puente entre la interfaz y la terminal: ejecuta los mismos comandos de la sección anterior con el módulo `subprocess` de Python, y después lee los tres archivos `.txt` que produce el propio programa en C –la misma fuente de verdad que si se ejecutara a mano–. En Windows usa las herramientas integradas junto al `.exe` (`win_flex` y `tcc`); en Linux y macOS, las del sistema. Si `lexer.l` se modifica después de la última compilación, la interfaz lo detecta por la fecha de los archivos y pide volver a compilar antes de ejecutar.

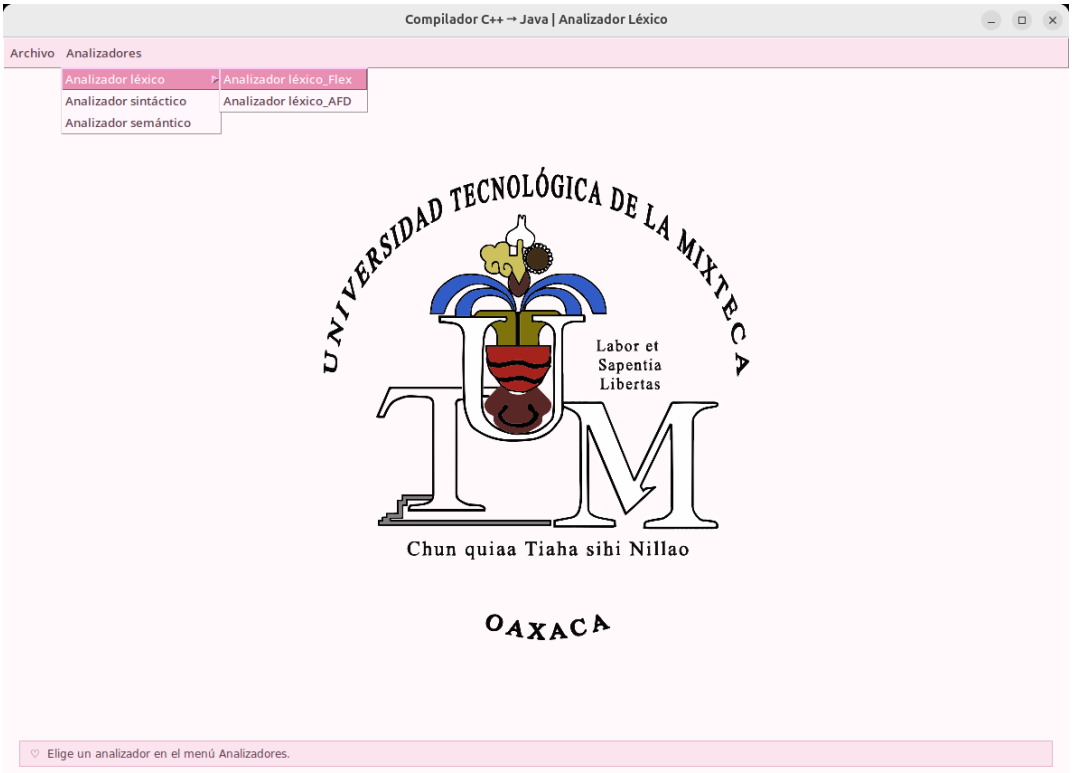


Figura 2: Pantalla principal con el menú *Analizadores* desplegado.



Figura 3: Panel *Analizador en Flex* en la interfaz, con `ErroresLexicos.cpp` cargado.

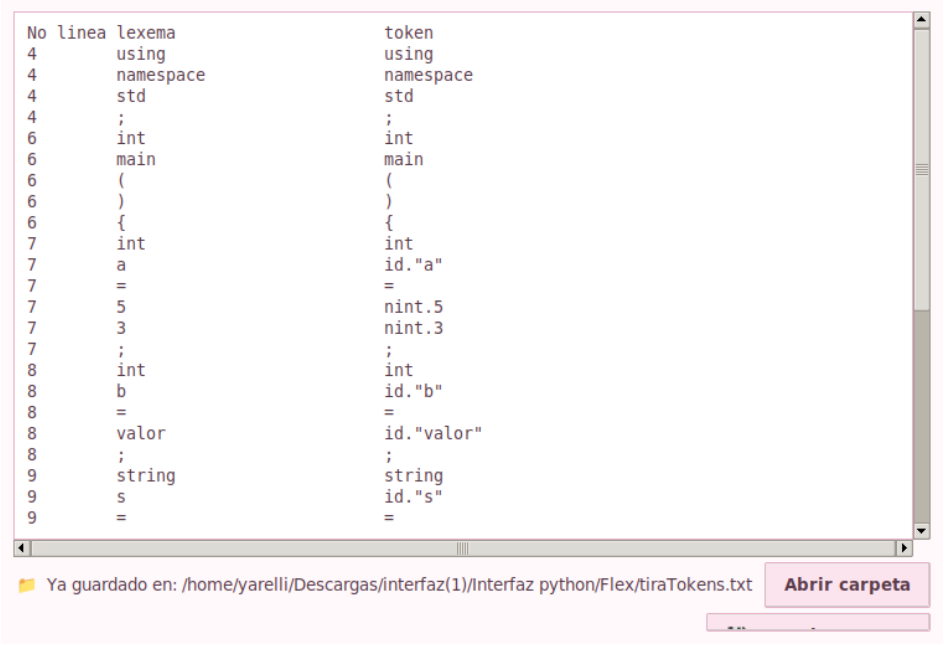


Figura 4: Ventana *Tira de tokens (Flex)* para `ErroresLexicos.cpp`.



Figura 5: Ventana *Tabla de símbolos (Flex)*: los seis identificadores del archivo, en orden de aparición.

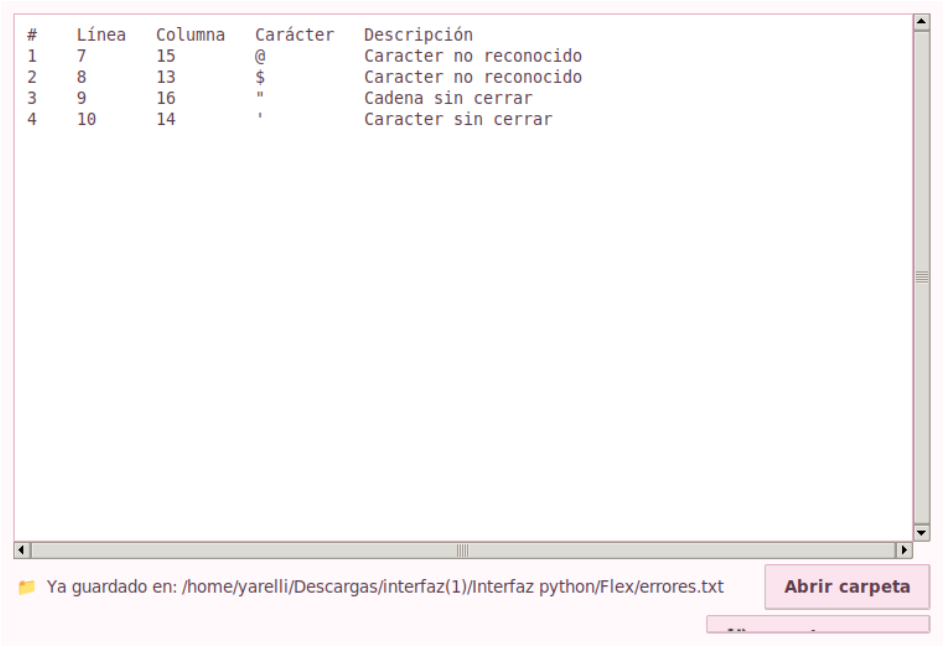


Figura 6: Ventana *Tabla de errores (Flex)*: los cuatro errores de `ErroresLexicos.cpp`.

7 Pruebas

Se ejecutó el analizador en Flex, ya compilado, sobre los mismos archivos de prueba de la primera entrega. La Tabla 8 resume los resultados reales de esa ejecución.

Tabla 8: Resultados del analizador en Flex sobre los archivos de prueba.

Archivo	Líneas	Tokens	Símbolos	Errores
`HolaMundo.cpp`	6	19	0	0
`Variables.cpp`	22	98	16	0
`Operadores.cpp`	54	211	4	0
`Comentarios.cpp`	18	43	4	0
`Calculadora.cpp`	36	147	7	0
`Fibonacci.cpp`	19	81	4	0
`Estructuras.cpp`	62	234	19	0
`DobleApuntador.cpp`	33	157	12	0
`prueba.cpp`	15	47	4	0
`ErroresLexicos.cpp`	14	41	6	4

Para verificar la implementación, se compararon línea por línea los tres archivos de salida de Flex con los que exporta la versión en Python para los mismos programas. Los resultados fueron idénticos en todos los casos, incluido `DobleApuntador.cpp`, que ejercita específicamente el token `**` en declaraciones como `int **ptr;` (Tabla 2), y `ErroresLexicos.cpp`, que ejercita los cuatro tipos de error de la Tabla 7.

Responsables por módulo de `lexer.l`

Tabla 9: Módulos de `lexer.l`, funciones y responsables.

Módulo	Funciones	Responsable	Actividad
Punto de entrada	`main()`, `YY_USER_ACTION`	Arenas Rojas I.	Abre el archivo fuente y los tres archivos de salida; sincroniza línea y columna antes de cada token; cierra los buffers al final.
Tira de tokens	`escribirToken()`, `escribirTokenValor()`	Bautista Jiménez I.	Formatea e imprime cada token reconocido, en consola y en `tiraTokens.txt`.

Módulo	Funciones	Responsable	Actividad
Errores léxicos	`escribirError()`, `BEGIN(RESTO_LINEA)`, `BEGIN(RESTO_ARCHIVO)`	Hernández Torija I.	Registra cada anomalía con su línea, columna, carácter y mensaje; recupera el análisis descartando el resto de la línea o del archivo.
Tabla de símbolos	`struct ElementoTSimb`, `insertar()`, `buscar()`, `agregarSimbolo()`, `imprimirTablaSimbolos()`	Juárez Jiménez Y. S.	Mantiene la lista enlazada de identificadores únicos, en orden de aparición.
Reglas del autómata	Definiciones (`letra`, `digito`, `identificador`, `nint`, `nfloat`, `escape`, `literalCadena`, `literalCar`) y reglas de números, cadenas, caracteres y comentarios	López López X. N.	Traduce la especificación léxica a expresiones regulares de Flex.
Clasificación de tokens	Reglas de palabras reservadas, operadores y delimitadores	Quiroz Martínez M.	Asigna la etiqueta de token a cada categoría léxica y ordena las reglas para preservar la precedencia entre operadores.

Conclusiones

Flex permitió implementar el analizador léxico a partir de las mismas expresiones regulares de la especificación, sin programar el autómata a mano. La herramienta aplica automáticamente las construcciones de Thompson y de subconjuntos que se estudian en el curso, y resuelve las ambigüedades con las mismas dos reglas que el equipo ya había implementado a mano en Python: coincidencia más larga y prioridad por orden de aparición en el archivo.

Comparar ambas versiones sirvió, además, para validar la primera entrega: los dos analizadores producen exactamente la misma tira de tokens, la misma tabla de símbolos y la misma tabla de errores sobre los diez archivos de prueba. Las condiciones de arranque exclusivas de Flex (`RESTO_LINEA`, `RESTO_ARCHIVO`) resultaron una forma natural de expresar la recuperación ante errores, y permitieron además detectar un caso que la versión en Python no reportaba: el comentario de bloque sin cerrar.

Trabajar con un generador de analizadores léxicos, en lugar de programar el AFD a mano, mostró en la práctica la diferencia entre *especificar* un lenguaje mediante expresiones regulares y *ejecutar* esa especificación: el tamaño del AFD que Flex construye (201 estados) creció respecto al de JFLAP (60 estados) porque cada palabra reservada se codifica como un camino propio del autómata, un costo que la primera entrega evitaba consultando una tabla hash después de reconocer un identificador genérico.

Recursos

Enlace de Google Drive con las actividades realizadas por el equipo: *informeActividadesEq3*.

Referencias

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas* (2.ª ed.). Pearson Educación.
2. Levine, J. (2009). *flex & bison: Text Processing Tools*. O'Reilly Media.
3. Paxson, V., Estes, W., & Millaway, J. *Lexical Analysis With Flex* (manual de Flex 2.6). https://westes.github.io/flex/manual/
4. The Flex Project. *flex: The Fast Lexical Analyzer Generator* (código fuente y documentación). https://github.com/westes/flex
5. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introducción a la teoría de autómatas, lenguajes y computación* (3.ª ed.). Addison-Wesley / Pearson Educación.
6. Louden, K. C. (2004). *Construcción de compiladores: Principios y práctica*. Cengage Learning.
7. WinFlexBison Project. *Flex and Bison for Microsoft Windows*. https://github.com/lexxmark/winflexbison
8. Bellard, F. *Tiny C Compiler (TinyCC)*. https://bellard.org/tcc/